

Task 1 — Iris Flower Classification

Notebook Link:

https://colab.research.google.com/drive/1xWxquVL94_me8EdFhxwID8tolgtb6HC6?usp=sharing

Summary:

Objective

The objective of this project is to build a machine learning model that can classify iris flowers into different species based on their sepal and petal measurements.

Dataset Used

The Iris dataset containing features such as:

- Sepal Length
- Sepal Width
- Petal Length
- Petal Width
- Species

Algorithms Used

- Logistic Regression
- K-Nearest Neighbors (KNN)
- Decision Tree Classifier

Steps Performed

- Imported and analyzed the dataset
- Checked for missing values
- Performed Exploratory Data Analysis (EDA)
- Visualized data using graphs and plots
- Encoded target labels
- Split dataset into training and testing sets
- Trained multiple machine learning models
- Compared model accuracies
- Generated confusion matrix and classification report
- Performed prediction on sample input
- Saved trained model using Joblib

Visualizations Included

- Pairplot
- Pie Chart
- Scatter Plots

- Box Plot

Evaluation Metrics

- Accuracy Score
- Precision
- Recall
- F1-Score
- Confusion Matrix

Result

Among the trained models, Logistic Regression achieved high accuracy in classifying iris flower species. The model successfully predicted flower categories based on input features.

Tools and Technologies Used

- Python
- Pandas
- NumPy
- Matplotlib
- Seaborn
- Scikit-learn
- Google Colab

Screenshots:

+ < > + T

Connect ^

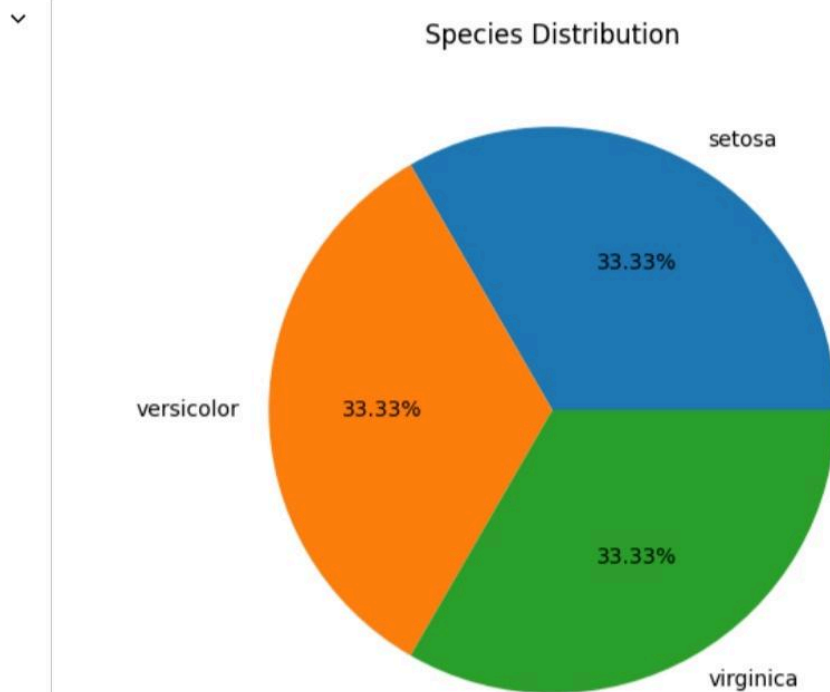
	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

```
[ ] sns.pairplot(df, hue='species', height=1.5)
plt.show()
```

> [Show hidden output](#)

```
[ ] df['species'].value_counts().plot(
    kind='pie',
    autopct='%0.2f%%',
    figsize=(6,6)
)

plt.title("Species Distribution")
plt.ylabel("")
plt.show()
```

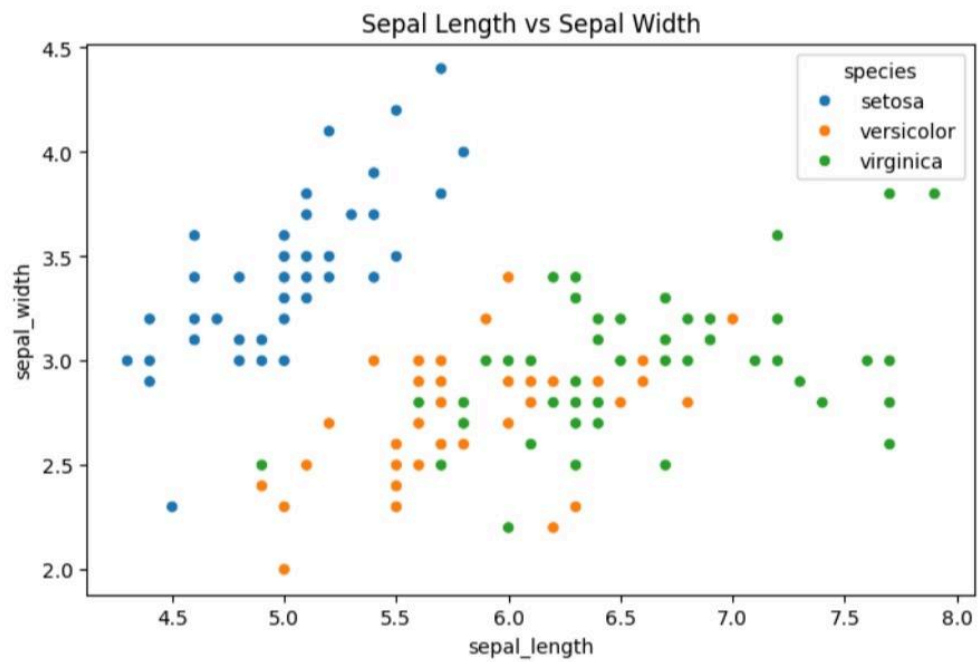


```
[ ] plt.figure(figsize=(8,5))

sns.scatterplot(
    data=df,
    x='sepal_length',
    y='sepal_width',
    hue='species'
)

plt.title("Sepal Length vs Sepal Width")
```

v



[]

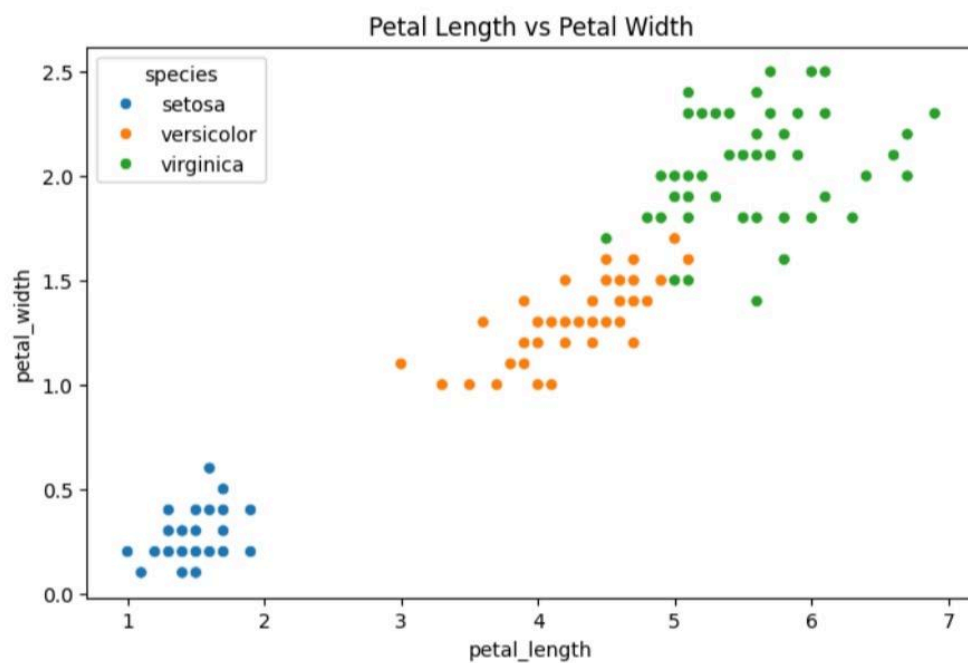
```
plt.figure(figsize=(8,5))

sns.scatterplot(
    data=df,
    x='petal_length',
    y='petal_width',
    hue='species'
)

plt.title("Petal Length vs Petal Width")

plt.show()
```

v

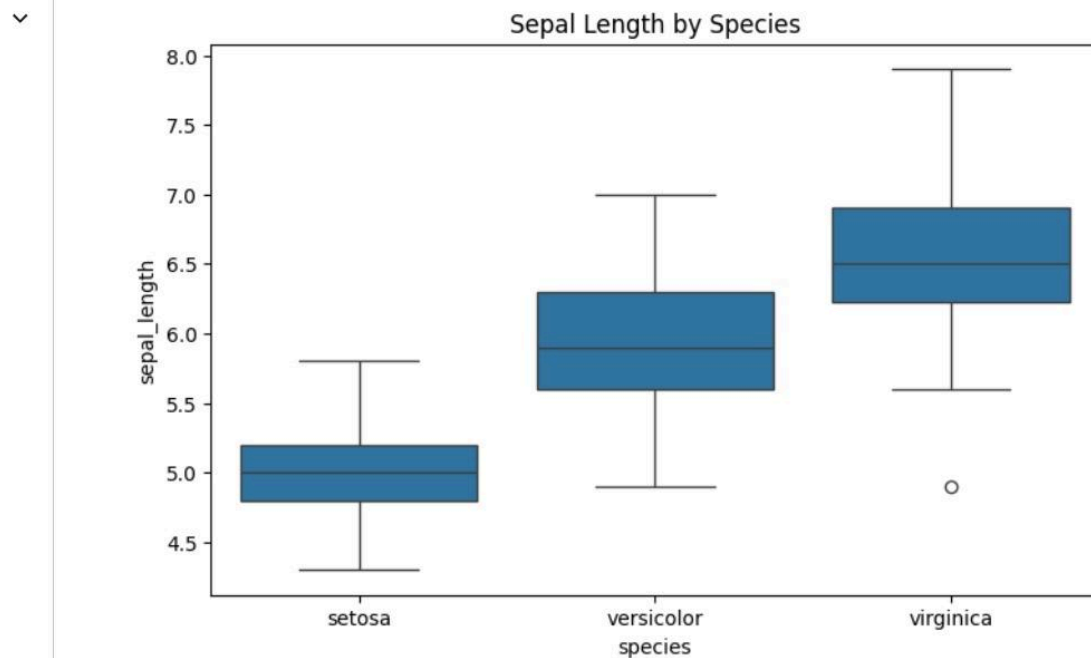


[]

```
plt.figure(figsize=(8,5))

sns.boxplot(
    data=df,
    x='species',
```

plt.show()



```
[ ] le = LabelEncoder()
df['species'] = le.fit_transform(df['species'])
df.head()
```

```

sepal_length sepal_width petal_length petal_width species
0           5.1         3.5         1.4         0.2         0
1           4.9         3.0         1.4         0.2         0
2           4.7         3.2         1.3         0.2         0
3           4.6         3.1         1.5         0.2         0
4           5.0         3.6         1.4         0.2         0

```

```
[ ] X = df.iloc[:, :-1]
y = df['species']
```

```
[ ] X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

```
[ ] log_model = LogisticRegression()
log_model.fit(X_train, y_train)
log_pred = log_model.predict(X_test)
log_acc = accuracy_score(y_test, log_pred)
print("Logistic Regression Accuracy:", log_acc)
```

```

Logistic Regression Accuracy: 1.0

```

+ <> + T

Connect ^

```
[ ] print(knn_acc, knn_acc)

print("Decision Tree:", dt_acc)
```

Model Accuracy Comparison

Logistic Regression: 1.0
KNN: 1.0
Decision Tree: 1.0

```
[ ] print("CLASSIFICATION REPORT")

print(classification_report(y_test, log_pred))
```

CLASSIFICATION REPORT

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	1.00	1.00	1.00	9
2	1.00	1.00	1.00	11
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30

```
[ ] cm = confusion_matrix(y_test, log_pred)

print(cm)
```

[[10 0 0]
 [0 9 0]
 [0 0 11]]

```
[ ] joblib.dump(log_model, "best_iris_model.pkl")

print("Best Model Saved Successfully")
```

Best Model Saved Successfully

```
[ ] sample = pd.DataFrame(
    [[5.1, 3.5, 1.4, 0.2]],
    columns=X.columns
)

prediction = log_model.predict(sample)
predicted_species = le.inverse_transform(prediction)

print("Predicted Species:", predicted_species[0])
```

Predicted Species: setosa

```
[ ] files.download("best_iris_model.pkl")
```

✓

+ <> + T

Connect ^

```
sepal_length sepal_width petal_length petal_width species
0           5.1         3.5         1.4         0.2  Iris-setosa
1           4.9         3.0         1.4         0.2  Iris-setosa
2           4.7         3.2         1.3         0.2  Iris-setosa
3           4.6         3.1         1.5         0.2  Iris-setosa
4           5.0         3.6         1.4         0.2  Iris-setosa
```

```
[ ] df.sample(5)
```

```
sepal_length sepal_width petal_length petal_width species
81           5.5         2.4         3.7         1.0  Iris-versicolor
77           6.7         3.0         5.0         1.7  Iris-versicolor
1            4.9         3.0         1.4         0.2   Iris-setosa
22           4.6         3.6         1.0         0.2   Iris-setosa
140          6.7         3.1         5.6         2.4  Iris-virginica
```

```
[ ] df.info()
```

```
<class pandas.core.frame.DataFrame>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   sepal_length    150 non-null   float64
1   sepal_width     150 non-null   float64
2   petal_length    150 non-null   float64
3   petal_width     150 non-null   float64
4   species         150 non-null   object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
```

```
[ ] df.describe()
```

```
sepal_length sepal_width petal_length petal_width
count      150.000000    150.000000    150.000000    150.000000
mean         5.843333         3.054000         3.758667         1.198667
std          0.828066         0.433594         1.764420         0.763161
min          4.300000         2.000000         1.000000         0.100000
25%          5.100000         2.800000         1.600000         0.300000
50%          5.800000         3.000000         4.350000         1.300000
75%          6.400000         3.300000         5.100000         1.800000
max          7.900000         4.400000         6.900000         2.500000
```

```
[ ] df.isnull().sum()
```

```
0
sepal_length 0
sepal_width  0
```